

Rethinking Thread Scheduling under Oversubscription: A User-Space Framework for Coordinating Multi-runtime and Multi-process Workloads

Aleix Roca

Barcelona Supercomputing Center
Barcelona, Spain
arocanon@bsc.es

Vicenç Beltran

Barcelona Supercomputing Center
Barcelona, Spain
vbeltran@bsc.es

Abstract

The convergence of high-performance computing (HPC) and artificial intelligence (AI) is driving the emergence of increasingly complex parallel applications and workloads. These workloads often combine multiple parallel runtimes within the same application or across co-located jobs, creating scheduling demands that place significant stress on traditional OS schedulers. When oversubscribed (there are more ready threads than cores), OS schedulers rely on periodic preemptions to multiplex cores, often introducing interference that may degrade performance. In this paper, we present: (1) The User-space Scheduling Framework (USF), a novel seamless process scheduling framework completely implemented in user-space. USF enables users to implement their own process scheduling algorithms without requiring special permissions. We evaluate USF with its default cooperative policy, (2) SCHED_COOP, designed to reduce interference by switching threads only upon blocking. This approach mitigates well-known issues such as Lock-Holder Preemption (LHP), Lock-Waiter Preemption (LWP), and scalability collapse. We implement USF and SCHED_COOP by extending the GNU C library with the nOS-V runtime, enabling seamless coordination across multiple runtimes (e.g., OpenMP) without requiring invasive application changes. Evaluations show gains up to 2.4x in oversubscribed multi-process scenarios, including nested BLAS workloads, multi-process PyTorch inference with LLaMA-3, and Molecular Dynamics (MD) simulations.

CCS Concepts: • Software and its engineering → Multi-processing / multiprogramming / multitasking; Scheduling; Application specific development environments.



This work is licensed under a Creative Commons Attribution 4.0 International License.

PPoPP '26, Sydney, NSW, Australia

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2310-0/2026/01

<https://doi.org/10.1145/3774934.3786451>

Keywords: High Performance Computing, Thread Oversubscription, Runtime Systems, User-Space Scheduling

ACM Reference Format:

Aleix Roca and Vicenç Beltran. 2026. Rethinking Thread Scheduling under Oversubscription: A User-Space Framework for Coordinating Multi-runtime and Multi-process Workloads. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '26)*, January 31 – February 4, 2026, Sydney, NSW, Australia. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3774934.3786451>

1 Introduction

The convergence of HPC, data analytics, and artificial intelligence (AI) is transforming HPC workloads [17, 20, 26, 34, 55]. Instead of monolithic applications, modern HPC workflows integrate multiple libraries and parallel programming models. This has significantly increased the complexity of execution environments, where multiple parallel applications may coexist and interact, leading to severe scheduling challenges.

HPC applications have traditionally relied on runtime systems to schedule parallel workloads on increasingly complex compute nodes with large multicore processors, NUMA architectures, and accelerators [1, 2, 7, 45, 51]. These runtime systems are responsible for mapping application parallelism (parallel loops and tasks) to the underlying hardware while improving performance and mitigating bottlenecks. Runtimes typically leverage low-latency synchronization mechanisms such as busy-wait barriers and create one thread per core to avoid oversubscription.

Under oversubscription, the OS scheduler enforces fairness with time-sharing policies, where threads are preempted based on fixed time quanta. While controlled oversubscription has been shown to be beneficial in some cases [3, 25, 28], the OS lack of application awareness, generally leads to scheduling noise, disrupts critical-path computations, and pollutes caches [12, 56].

Runtime composition poses a significant challenge in HPC [16, 29, 52]. When multiple runtime systems are used within the same process, they each create independent worker pools, leading to thread redundancy and resource

contention. Moreover, since runtimes assume exclusive system access and use aggressive synchronization techniques, preemptions severely degrades their performance. In consequence, developers have historically avoided combining runtimes—not because it lacks utility, but because it typically causes oversubscription and poor performance. However, as HPC applications grow more complex and integrate heterogeneous software stacks, strict component isolation is becoming impractical.

Similarly, HPC applications are often run in exclusive mode to avoid interferences from other workloads and ensure predictable and efficient execution. Despite this, many applications exhibit varying degrees of parallelism during execution. These workloads transition between parallel phases and sequential phases, where underutilization of system resources occurs due to inherent algorithmic constraints. However, freely coexecuting processes in the same node to “fill the gaps” is discouraged because of the detrimental effects of oversubscription.

Existing solutions to mitigate oversubscription generally rely on resource partitioning [5, 60, 64, 68] or using a single unified runtime for all involved components [18, 35, 47]. However, the vast heterogeneity of HPC software stacks makes broad adoption of these solutions difficult, as competing APIs introduce additional complexity. Given these challenges, we believe that a transparent approach is necessary to facilitate adoption in the HPC community.

In this paper, we propose USF, a User-space Scheduling Framework with multi-process support that enables the creation of ad-hoc user scheduling policies. Unlike similar frameworks [18, 27, 30, 47, 73], USF requires no application modifications, no special permissions, works on vanilla Linux kernels, operates entirely in user space, and seamlessly supports Thread-Local Storage (TLS). We leverage USF to implement SCHED_COOP, a seamless cooperative policy designed to mitigate oversubscription in HPC environments. SCHED_COOP builds on the basis that HPC applications benefit from long uninterrupted runs with minimum interference. Applications scheduled by SCHED_COOP are not periodically preempted, but instead are only swapped upon blocking. We implement both USF and SCHED_COOP completely in user-space by extending the GNU C library and, in particular, its pthread API. Our solution supports any application that relies on the pthread synchronization APIs provided by glibc, covering the vast majority of runtime systems—such OpenMP (gomp, libomp), pthreadpool (PyTorch), or Rust.

Our evaluation demonstrates that (1) customized seamless user-space schedulers have potential to improve current scheduling limits, and (2) SCHED_COOP is effective at mitigating oversubscription issues while preserving available parallelism. By minimizing thread preemptions and context-switching overhead, our policy improves application throughput and system efficiency. Since our framework

operates on top of the Linux scheduler, critical OS services and non-cooperative threads remain unaffected.

The main contributions of this paper include:

- USF, a seamless user-space scheduling framework that allows users to create their own scheduling policies.
- SCHED_COOP, a USF cooperative scheduling policy to address oversubscription in HPC and AI workloads.
- A detailed evaluation in multi-runtime and multi-process execution scenarios.
- A practical approach to runtime nesting that avoids severe performance penalties.
- Guidelines for tuning oversubscribed applications.

2 Background

This section details the main components of USF and SCHED_COOP, implemented by extending glibc with the nOS-V [74] threading and tasking library.

2.1 The Linux Kernel Process Scheduler

The Linux kernel process scheduler is the ultimate authority in deciding which threads run on which cores. The scheduler is implemented in layered classes with increasing priority levels [10]. The default class is the Earliest Eligible Virtual Deadline First (EEVDF) scheduler [9, 54]. This policy attempts to schedule ready threads fairly, assigning each a quantum and preempting them if oversubscribed. Preemptive schedulers, such as EEVDF, can guarantee progress, but because preemptions happen without prior notice, they might interrupt critical operations and harm the performance of sensitive applications. The Linux scheduler exposes few configuration options to user-space, which limits the user’s ability to influence scheduling decisions.

2.2 The GNU C Library

The GNU C Library [21] is one of the most common libraries in the Linux ecosystem. glibc provides the C runtime, the Linux dynamic loader, core C APIs (pthreads, I/O, math, etc.), and syscall wrappers. Its backward compatibility makes it the default choice for most compilers, which link it to all C applications by default. Therefore, glibc is the ideal convergence point to intercept process initialization, thread creation, blocking operations, affinity management, and, at the same time, transparently cover a wide range of applications. We implemented USF and SCHED_COOP inside glibc and named it glibc_v.

2.3 nOS-V

nOS-V [74] is a flexible threading and tasking library with multi-process support that we use at the core of glibc_v as detailed in Section 4. While nOS-V is fully documented in its original paper, we briefly summarize the key details here.

nOS-V provides bare-bones services for runtime systems. The nOS-V library defines the concept of task and its API

provides the means to schedule and execute them in worker threads, which are managed by nOS-V transparently. At a glance, the nOS-V API allows to create tasks, submit them to the scheduler, run callbacks upon task completion, pause tasks, and resume them. By leveraging these fundamental functionalities, it is possible to build a full parallel runtime system with dependency management and standard features (e.g. `taskwait`), such as NODES [39] (task-based) or libompv [61] (task and fork-join). Hence, nOS-V can be seen as a library to help build runtimes.

nOS-V initializes a worker pool and always keeps exactly one running worker pinned per core at all times. Idle workers wait until the centralized scheduler assigns them a task, which they run to completion. However, tasks can suspend themselves while waiting on user-defined conditions (e.g., synchronization points like `taskwait`). When this happens, nOS-V performs a worker swap, replacing the suspending worker with another to keep the core busy. Once the wait condition is met, the task is queued into the scheduler and later assigned to an idle worker. If a worker is serviced a task already bound to a different worker, nOS-V performs another swap so both the original worker and its task resume on the newly allocated core. nOS-V provides several algorithms to select the next task to run; we detail the strategy used in SCHED_COOP in Section 4.1.

A key nOS-V feature is seamless multi-process scheduling. Internally, nOS-V relies on a shared memory segment [36] to keep tasks, workers, and other relevant structures. Each process creates its own tasks, but a single instance of the centralized scheduler manages all tasks from all processes. Idle workers can be handed tasks belonging to any process but only execute their own. If a worker receives a task belonging to another process, the worker swaps itself with a worker of that process and transfers its task to it. nOS-V allocates a configurable quantum per-process but never preempts involuntarily. Instead, process switches occur only at scheduling points (task suspension, end, or yield).

3 SCHED_COOP

From the user’s perspective, SCHED_COOP resembles a Linux scheduling policy like SCHED_RR (real-time round-robin). Unlike those, however, it is a **virtual** scheduler implemented entirely in user space via USF.

Processes enter the SCHED_COOP at launch time with the new “`chrt -c <app>`” command option. Child processes and threads inherit it by default at runtime, and only exit it at termination time. SCHED_COOP threads run uninterruptedly with a fixed single-core affinity until the application conditions (not OS conditions) compel it to wait. Only when a thread blocks or voluntarily relinquishes its core, another thread might be resumed on it.

Although SCHED_COOP threads run greedily, they do not stall non-SCHED_COOP processes, since the default OS

scheduler may still preempt them. The key distinction is that SCHED_COOP threads never preempt one another. In this sense, SCHED_COOP resembles SCHED_RR, where threads run until they yield or block. However, unlike SCHED_COOP, SCHED_RR threads cannot be preempted by regular threads (only by higher-priority SCHED_RR threads), which risks stalling the system; hence SCHED_RR is restricted to root and system users. By contrast, SCHED_COOP is available to regular users and, at worst, its threads may stall only other SCHED_COOP threads.

The SCHED_COOP goal is to improve throughput and system efficiency by improving scheduling decisions in oversubscription scenarios. Therefore, it is not intended for all workloads, but for parallel applications or multi-process workloads leveraging multiple runtime systems prone to oversubscription.

4 Design

This section follows a three-layered top-down approach. Section 4.1 explains how USF and SCHED_COOP operate from a threading perspective. Section 4.2 describes how this behavior is achieved by coupling glibc with nOS-V. Section 4.3 delves deeper into the extended glibc features and the nOS-V API usage, and Section 4.4 discusses this implementation limitations.

4.1 USF and SCHED_COOP Threading Overview

USF works seamlessly by extending standard glibc APIs. At a glance, it manipulates thread creation, destruction, and blocking APIs. Pthread creation is extended to prevent threads from running freely. Instead, they are blocked and queued until an idle core is available. A centralized scheduler monitors available queued threads and dispatches them to idle cores according to the current policy. Threads resume their execution pinned to an idle core and run uninterruptedly until they end, block, or yield. When a thread reaches a blocking operation (e.g., a contended `pthread_mutex_lock`), it notifies the scheduler that its core is about to become idle. In response, the scheduler pins and resumes another thread on the idling core. Unblocking paths (e.g., `pthread_mutex_unlock`) notify the scheduler of threads ready to be resumed, but these threads are not resumed immediately. Instead, they are queued within the scheduler.

SCHED_COOP is a policy that defines a simple algorithm to select the next thread to run. By default, all threads have a preferred affinity set to the last core on which they ran. Once a previously blocked thread becomes ready, it is queued in a per-process, per-core First-In First-Out (FIFO) queue. The scheduler first assigns threads to idle cores matching their affinity; if none are available, it searches within the same NUMA domain, and otherwise assigns them to any other NUMA node. When the current process’ quantum expires (20 ms by default), evaluated at task scheduling points, the

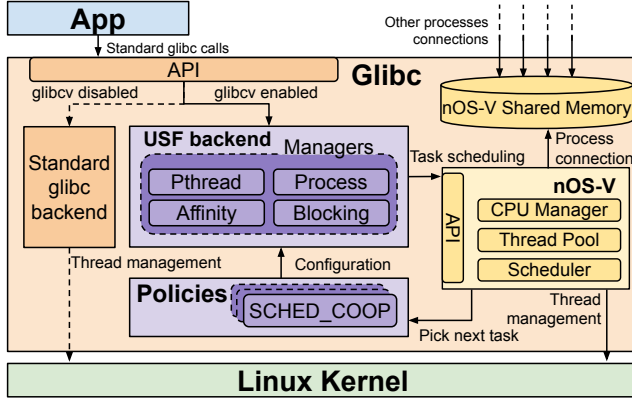


Figure 1. Glibcv architecture diagram. Application’s standard API calls are forwarded to the USF backend if enabled, which bridges with the nOS-V API. nOS-V schedules threads according to the selected policy.

scheduler continues assigning tasks from the next process’s queues. The scheduling logic is part of nOS-V, as explained in Section 2.3.

4.2 glibcv: Coupling glibc and nOS-V

Figure 1 shows the glibcv architecture diagram. Both USF and SCHED_COOP are implemented within glibc, and they both rely on the nOS-V threading library to manage most of the scheduler logic described before. Conceptually, we use nOS-V to convert glibc into a task-based runtime; all pthreads become worker threads with an associated task, and most synchronization APIs become task block/resume operations. Since each task remains bound to the same worker thread, they are fully compatible with TLS.

When a pthread is created, glibcv transparently converts it into a nOS-V worker thread and assigns it a new task mapped to the pthread’s entry function. The task is then submitted to the nOS-V scheduler and later resumed on an idle core with a fixed single-core affinity, where it runs until the user code ends, blocks, or yields. When the user code ends, the task ends, and nOS-V schedules another worker in its core. On blocking calls such as `pthread_mutex_lock`, the task suspends and nOS-V also schedules another worker in that core. On unblocking calls, such as `pthread_mutex_unlock`, the suspended task is resubmitted to the scheduler. Forked processes and their threads are also registered within nOS-V, which manages all tasks in a single multi-process centralized scheduler.

Note that because we prevent threads from running freely and nOS-V carefully keeps a single thread per core, threads must run in an orderly manner, and we avoid preemptions between USF threads. See Section 2.3 for more details on nOS-V scheduling.

4.3 Extended API Calls

In this section, we explain the implementation details of the extended glibc APIs with nOS-V, as depicted in the Figure 1 USF backend block.

4.3.1 Pthread Creation. Whenever `pthread_create()` is called, glibcv swaps the user-provided function pointer with a wrapper. Once the thread is created, the wrapper calls the `nosv_at tach()` API, which registers the calling thread as a nOS-V worker, assigns the thread’s entry function to a new nOS-V task, assigns the task to the worker, submits the task to the scheduler, and immediately blocks the current thread. At this point, glibcv has created a pthread following the provided user attributes (except for the affinity), but it has recruited it as a nOS-V worker, which means that it can no longer run freely. We have extended the `pthread_t` object to accommodate the nOS-V task object and related data.

USF uses a thread caching strategy, similar to that proposed by D. Dice and A. Kogan [13], to avoid the overhead of repeatedly creating and destroying threads. When a thread’s user function ends (or calls `pthread_exit()`, unwinding the stack), the thread blocks and is swapped out with another worker instead of exiting. If another thread calls `pthread_join`, the join operation is masked and, instead, the completed thread is placed in the cache. Subsequent `pthread_create` calls reuse the most recent cached threads rather than creating new ones. At shutdown, all cached threads are unblocked, detached, destroyed, and truly joined. Threads are detached by calling `nosv_detach()`, which deregisters them from nOS-V and terminates their task.

4.3.2 Affinity Management. Attempts to change thread affinity (e.g., `pthread_setaffinity_np()`) under USF are considered hints. This is necessary to prevent interfering with the nOS-V fine thread placement. In such cases, glibcv stores the user affinity in the `pthread_t` object, but does not change it. To preserve application compatibility, when queried with `pthread_getaffinity_np()` and similar APIs, glibcv returns the previously stored affinity instead of the real affinity. For the same purpose, we have also extended the `syscall()` system call entry point to intercept `sched_setaffinity`, which is often called directly because it has different semantics than the glibc wrapper. We also use hash tables to keep track of tid-task mapping, necessary for the `sched_set/getaffinity` calls. However, if needed, it would be simple to create a USF policy that respects the user affinity.

4.3.3 Forks and Process Initialization. Glibcv can manage multiple processes. At process startup, if the `USF_ENABLE` environment variable is set, glibcv initializes nOS-V before any constructor is called. As explained in Section 2.3, nOS-V connects to any pre-existing shared memory segment and registers the new process. Then, glibcv converts the current

Listing 1. Glibc mutex lock/unlock extension. When a mutex is contended, threads are placed in a spinlock-protected per-mutex FIFO wait queue before blocking in nOS-V. On unlock, if the queue is not empty, one thread is dequeued and submitted to the scheduler.

```
int pthread_mutex_lock(
    pthread_mutex_t *mutex) {
    nosv_task_t task = pthread_self()->task;
    pthread_spin_lock(&mutex->spinlock);
    if (pthread_mutex_trylock(mutex)) {
        // Contended
        queue_add_tail(&mutex->queue, task);
        pthread_spin_unlock(&mutex->spinlock);
        nosv_pause();
    } else {
        pthread_spin_unlock(&mutex->spinlock);
    }
    return 0;
}

int pthread_mutex_unlock(
    pthread_mutex_t *mutex) {
    pthread_spin_lock(&mutex->spinlock);
    list_head_t wait = &mutex->queue;
    nosv_task_t task = queue_pop_head(wait);
    if (!task)
        // Standard mutex unlock
        __pthread_mutex_unlock(mutex);
    pthread_spin_unlock(&mutex->spinlock);
    if (task)
        nosv_submit(task);
    return 0;
}
```

main thread into a nOS-V worker with an attached task using `nosv_attach()`. When the process terminates or when `exec` is called, glibc calls `nosv_shutdown()` to deregister the process and shutdown its worker threads. In the case of `exec`, after the operation completes, the new application also finds the `USF_ENABLE` environment variable and registers the process into nOS-V again.

4.3.4 Blocking APIs. We extended the mutex, condition variable, barrier, semaphore, sleep, yield, and polling (poll, epoll) APIs to detect blocking operations. Listing 1 shows the mutex pseudocode. We augmented `pthread_mutex_t` with a FIFO wait queue and a spinlock to protect it. On contention, `pthread_mutex_lock()` enqueues the calling thread's nOS-V task (each pthread is associated with one), and blocks it via `nosv_pause()`, which triggers a worker swap. On `pthread_mutex_unlock`, if contended tasks exist,

one is submitted to the nOS-V scheduler (`nosv_submit()`) and ownership is transferred to that task; only if no tasks are queued is the mutex fully released.

Other APIs follow the same pattern: contended tasks are tracked in standard glibc objects and managed with nOS-V calls to block, resume, yield, or wait. Timed variants of poll and epoll periodically perform non-blocking event checks using the original API, then call `nosv_waitfor()` to perform a short timed wait in a loop (5ms by default) until either the user timeout expires or an event occurs. `nosv_waitfor()` also triggers a worker swap, but tasks are automatically resubmitted once the provided timeout elapses.

4.4 Limitations

USF efficient use of cores partly depends on its capacity to intercept wait operations, such as blocking operations. If a wait operation is not intercepted, nOS-V is unable to detect the idle core, and it cannot run other tasks on it until the operation completes. Instrumenting blocking operations is simple because these are likely to use standard glibc APIs or a kernel syscall wrapper. However, some applications or runtimes rely on busy-waiting barriers to implement low-latency synchronizations. These cases are complex to instrument at the glibc level, but failing to do so might stall progress and deadlock the application. This can occur if the number of waiting threads exceeds the number of cores or if multiple barriers are active simultaneously. Instead, preemptive schedulers mask this into a performance problem because they do not depend on scheduling points to guarantee progress in oversubscribed systems. As we further detail in Section 5.2, our solution to these cases is to manually adapt the busy-wait barriers to include an occasional yield operation (such as `sched_yield()`) every few busy iterations, which is not only beneficial for policies such as `SCHED_COOP`, but also for oversubscribed preemptive schedulers (see Section 5.3).

USF supports any application that relies on the glibc synchronization APIs. Most runtimes typically offer a POSIX-compatible backend alongside platform-specific alternatives for Windows, macOS, or Linux (e.g., custom `futex`-based implementations). Extending USF to support `futex`-based backends via glibc's `futex` wrapper is part of our planned future work.

USF policies inherit the nOS-V security limitations. nOS-V shares state via a shared memory segment. This shared memory is visible to all processes connected to the same nOS-V instance, and sensitive information could be exposed between them. For this reason, we only allow processes of the same user and group to connect to the same shared memory. In the future, we will study a tighter nOS-V integration with the Linux kernel to overcome this limitation.

Because glibc extends standard glibc objects that are statically allocated (such as `pthread_mutex_t`) and its size is calculated at compile-time, all applications and their dependencies must be rebuilt against glibc. Fortunately, build

Table 1. Evaluation machine specifications.

Machine	Marenostrum 5
Memory	256GiB
Network	InfiniBand (100Gb/s)
CPU Model	Intel Sapphire Rapids 8480+
GPU	-
CPU Arch.	x86_64
CPU Cores	56x2
L1, L2, L3	80KB, 2MB, 105MB

systems such as Nix [15] are capable of automating whole system builds, which greatly simplifies testing glibc with applications with complex dependencies, such as PyTorch.

5 Evaluation

In this section, we evaluate USF and SCHED_COOP across progressively more complex scenarios. We begin by outlining our evaluation methodology, which follows a bottom-up approach. Our first hypothesis is that a user-space scheduler, combined with manual adaptations to the software stack, can improve the performance of oversubscribed applications. Our second hypothesis is that similar gains can be achieved seamlessly, without invasive modifications. To validate these hypotheses, Section 5.1 introduces the evaluation scenarios—nested runtimes and multi-process coexecution—and describes the experimental setup. Then, Section 5.2 addresses interference considerations, ensuring fair comparisons by harmonizing runtime wait policies and disabling thread pinning. Section 5.3 validates both hypotheses with a controlled matrix multiplication benchmark, contrasting manual adaptations with the seamless USF approach. Section 5.4 extends the analysis to a Cholesky factorization benchmark with multiple runtime compositions, highlighting the challenges of heterogeneous software stacks. Next, Sections 5.5 and 5.6 demonstrate USF in a practical AI microservices and molecular dynamics scenarios, respectively, showcasing its benefits in realistic multi-process environments.

5.1 Evaluation Scenarios and Setup

We concentrate on two representative oversubscription scenarios within a single node: nested runtimes, where threads from an outer runtime invoke parallel regions in an inner runtime, causing the number of active threads to grow rapidly (e.g., an application with n oneTBB threads each calling a BLAS routine that spawns an OpenMP region with n threads, yielding up to n^2 threads); and multi-process workloads, where several parallel applications overlap sequential and parallel phases to maximize resource use. Our experiments compare performance under the standard Linux scheduler (baseline) and SCHED_COOP. We use the production system

described in Table 1, where kernel modifications are not allowed, demonstrating the value of our transparent user-space approach. Our software environment includes: Linux 5.14, glibc 2.40-6, nixpkgs 25.05, clang 20, gcc 14.2.1, OpenBLAS 0.3.28 [22], BLIS 1.1 [53, 59], PyTorch 2.6 [43], Transformers 4.51.3 [63], oneTBB 2020.3 [49], MPICH 4.3.0 [23], LAMMPS 29Aug2024_update2 [46, 57], DeePMD-kit 3.1.1 [62, 69, 70], nOS-V 3.2 [74], Nanos6 4.2 [6], and NODES 1.3 [39].

5.2 Interference Considerations

In order to achieve a fair comparison, we first consider reasonable parameters to runtimes and other libraries that would otherwise clutter the results concerning the differences between the Linux process scheduler and SCHED_COOP. These changes are necessary because most runtimes and some parallel libraries are not designed to coexist in the same address space. We have tested our applications with and without these adaptations and decided to ultimately include them because of their performance benefits for both the baseline and SCHED_COOP schedulers.

Wait policies are common to most runtimes, such as OpenMP (OMP_WAIT_POLICY) and OmpSs-2 [44]. They define the behavior of worker threads upon work starvation. By default, both runtimes assume a hybrid wait policy where workers embrace a timed active wait prior to taking the expensive OS block/resume path. However, a runtime worker actively waiting for work might be wasting CPU cycles and delaying the execution of another runtime worker with ready tasks to execute. In consequence, all of our experiments have their runtimes configured with similar "passive" wait policies (recommended under oversubscription), in which threads immediately take the OS blocking path instead of actively waiting for more work. Thread pinning is another feature that lets users configure if and how worker threads are pinned to which system cores (e.g., OMP_PLACES for OpenMP). However, our early evaluation showed that multiple runtimes or oversubscribed processes pinning their workers to overlapping CPU sets interfere with each other, dramatically hindering performance. All our tests avoid these problems by configuring runtimes to skip thread pinning.

Expanding our analysis beyond runtime systems, we observed a recurring pattern: some libraries implement custom busy-waiting barriers instead of relying on standard APIs. In systems with no more active threads than available cores, busy-waiting barriers can be an effective way to briefly stall threads. However, in oversubscribed systems, scheduling noise can significantly delay barrier completion. A single preempted thread can block an entire team, and worse, busy-waiting threads continue consuming CPU cycles until their time slice expires—potentially delaying the very thread needed to release the barrier. This issue is further exacerbated when multiple busy-waiting barriers operate

Listing 2. Simplified matmul code parallelized with runtime nesting: OmpSs-2 (Nanos6) and BLIS (OpenMP). The problem is subdivided in NB blocks, and each block is of size TS. The "oss task" pragma creates a task with the specified dependencies, and the "oss taskwait" blocks execution until all previous tasks have been completed.

```

for (k = 0; k < NB; k++)
  for (i = 0; i < NB; i++)
    for (j = 0; j < NB; j++)
      #pragma oss task inout(C[i][j]) \
        in(A[i][k]) in(B[k][j])
      cblas_dgemm(TS,
        A[i][k], B[k][j], C[i][j]);
#pragma oss taskwait

```

simultaneously, a scenario frequently encountered when parallel BLAS kernels execute concurrently. For our evaluation, we follow a common pattern: to yield within the busy-wait barriers to allow other threads to progress. We do so by modifying a single line of code in OpenBLAS, BLIS, and MPICH, to add the sched_yield() syscall (details in Section 5.3).

A side effect of nesting parallel runtimes is that the resulting kernel granularity might be too small. A nested parallel loop or similar structure gets subdivided into an exponential number of micro-kernels. Fine-grained kernels on runtime systems pose a challenge on its own [4, 8, 40, 66] because the runtime and system overheads become critical. In this study, however, we focus on the analysis of oversubscription, and we assume sufficiently large workloads.

5.3 Manual and Seamless Scheduling Comparison

In this section we evaluate our two hypotheses as defined in Section 5.1 using a simple Matrix Multiplication (matmul) benchmark. Our matmul (see Listing 2) is an example of the nested runtimes case, where we applied blocking to subdivide the problem with OmpSs-2 tasks (outer runtime) and process each block using BLIS parallelized with LLVM's OpenMP (inner runtime). Each OmpSs-2 task is executed by a Nanos6 thread, and each task opens an OpenMP parallel region within the BLAS library.

We fix all input matrix sizes to 32768×32768 and evaluate the benchmark response to oversubscription by varying the task size (outer runtime parallelism) and number of OpenMP threads (inner runtime parallelism). For example, in our 56-core system, with 2 OpenMP threads and 16384 task size (at most $(32768/16384)^2 = 4$ parallel tasks at the same time), we have a total of $2 \times 4 = 8$ threads busy at the same time and the system is mostly underused. With 56 OpenMP threads and 512 task size we have 229376 busy threads and the system is heavily oversubscribed. We run each kernel in a loop for at

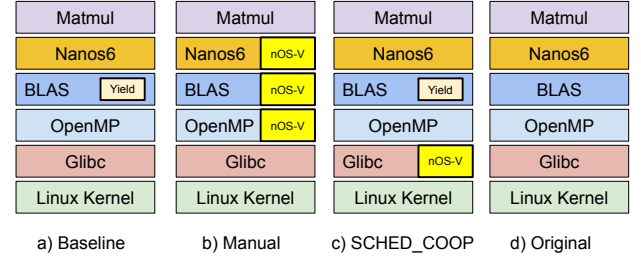


Figure 2. Evaluated matmul software stacks. a) Baseline with yield. b) Manual nOS-V integration. c) Seamless nOS-V integration. d) Unmodified (no yield).

least 60s and measure MOPS/s performance metric computed as $\frac{\text{size} \cdot \text{loops}}{\text{seconds}} \cdot 10^{-6}$.

We compare four matmul versions, shown in Figure 2, ordered from least to most invasive changes: Original is the unmodified implementation. Baseline is identical to Original but incorporates yielding on barriers (Section 5.2) and serves as our reference point. SCHED_COOP uses the same software stack as Baseline, except that glibc is replaced with glibc_v to enable our scheduler through nOS-V. The Manual version implements the same scheduling policy as SCHED_COOP, but instead of enabling it transparently through glibc_v, it is integrated manually via explicit calls to the nOS-V threading library within the Nanos6, OpenMP, and BLAS libraries. The goal of the Manual configuration is to establish an upper bound for glibc_v's performance by maximizing gains without the constraints of a seamless solution, incorporating minor ad-hoc optimizations at each library level. In essence, Manual attaches pthreads created at the different levels with nosv_attach and uses nOS-V blocking primitives instead of pthread primitives.

Figure 3 shows the results of the experiment. For each evaluated matmul version, the heatmap colors helps us to roughly distinguish three areas: The lower left triangular area corresponds to under-used resource configurations (fewer threads than cores and coarse tasks). The top-right triangular areas represent overly fine-grained blocks caused by exponential fragmentation (more threads than cores and fine-grained tasks). The middle diagonal area corresponds to oversubscribed configurations with reasonable granularities. We consider the upper and lower triangular areas to be ill-designed: either not enough or too much parallelism is exposed. Instead, our focus is on the middle areas, which include a large range of valid configurations. Note that the configurations with a single runtime enabled, i.e. (1-32768, *) and (*, 1), consider scenarios without runtime composition.

As per our first hypothesis, we compare Baseline with Manual (see Figure 3 a) and b), respectively) to demonstrate that the nOS-V user-space scheduler can improve the performance of an oversubscribed application. At first glance, the heatmap coloring reveals how the Manual version is capable

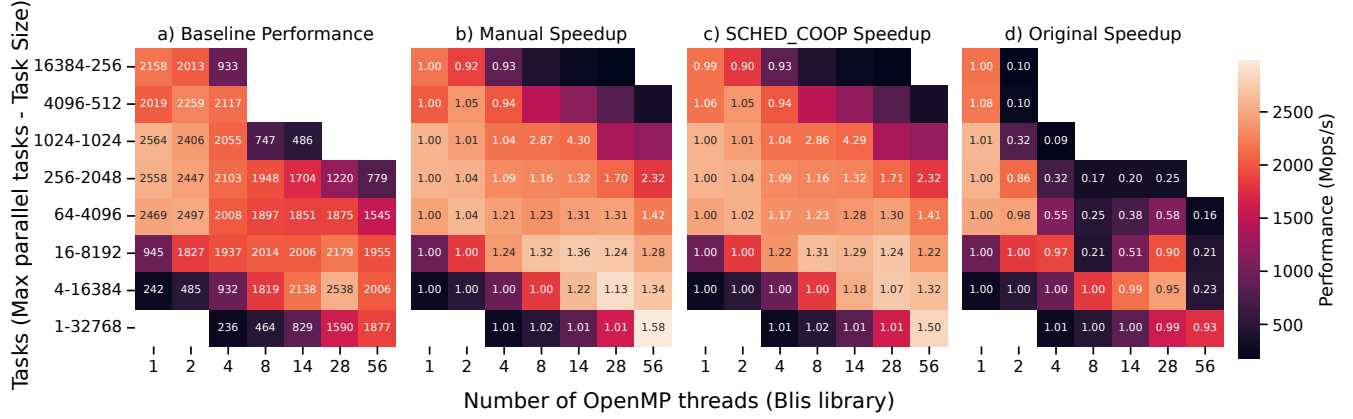


Figure 3. Evaluation of matmul with two nested runtimes. Figure a) numbers show performance (higher is better), while b), c), and d) show element-wise speedup with respect to a). The colors of all heatmaps show performance, not speedup. White squares are timed out experiments (>15 minutes). Squares without speedup lack the Baseline reference because of time out.

of expanding the "hotter" (more performant) regions with respect to Baseline, particularly in the middle area. Then, the speedups depicted in Manual detail an improvement between 1% and 36% depending on the configuration. Hence, we can conclude that the first hypothesis is valid. Note that although we achieve up to 4x speedup on some configurations, we do not consider them relevant because the resulting performance is low when compared to the best results obtained.

As per our second hypothesis, we compare Baseline with SCHED_COOP (see Figure 3 a) and c), respectively) to demonstrate that a seamless approach can also improve the performance when under oversubscription. By using the same method, we can see a similar performance improvement by comparing the color gradients of both heatmaps and observe a speedup between 2-28%. Thus, we consider our second hypothesis validated as well. Moreover, the best SCHED_COOP configuration (28,4-16384), which involves the two runtimes, improves the best Baseline configuration (1, 1024-1024), with a single runtime, by almost 9.8%. Also, the best Manual configuration (28,4-16384) improves the best Baseline by 11.77%. This shows that mixing runtimes in the same address space can be beneficial if under the appropriate scheduling policy.

In this experiment, the matmul with nOS-V versions obtain a performance increase mostly because of how BLAS busy-wait barriers are managed. Essentially, even if sched_yield is commonly used to relinquish the current core, Linux might not yield immediately but instead waits until the thread's quantum expires [3]. The details depend on the Linux version used, but its usage is still beneficial because threads yield as soon as possible instead of waiting for the next clock interrupt. For comparison, see the results of Original in Figure 3 d). This version is the same as Baseline, but it uses the original busy-wait barrier without sched_yield. We can see large slowdowns in many configurations, particularly

the most oversubscribed because most time is spent busy-waiting. Instead, the matmul SCHED_COOP version always yields because the original sched_yield uses a nOS-V yield API internally. Yielding sooner is beneficial for this scenario because it allows threads to exit barriers earlier when under heavy oversubscription.

In conclusion, both Manual and SCHED_COOP can improve the range of configurations that attain higher performance, with the advantage that SCHED_COOP can do it transparently. USF enables the implementation of tailored user-space scheduling policies without kernel modifications, such as SCHED_COOP. This has the potential to effectively unlock runtime composition and oversubscription scenarios, which so far had generally been avoided because of their penalties. It also eases the exploration of the input configuration space. Instead of searching for the right thread balance between runtimes, users can focus on finding configurations that expose enough parallelism while maintaining coarse-enough tasks.

5.4 Runtime Compositions

In this Section, we show how complex the runtime composition combinations can quickly become even if working with a relatively simple benchmark. To do so, this time we use a Cholesky factorization benchmark and multiple combinations of different runtime implementations, including GNU's gomp, LLVM's libomp, and Intel's tbb. Similar to matmul, our Cholesky first divides the problem into blocks using an outer runtime, then for each block, it calls a BLAS API that further subdivides and computes the kernel in parallel using an inner runtime (either OpenMP or pthread based). For brevity, we present results for a subset of relevant Baseline and SCHED_COOP configurations. We fix the input size and task size to 32768×32768 and 1024 respectively. The results are shown in Table 2. SCHED_COOP outperforms

Table 2. Cholesky evaluation results combining multiple run-times for three degrees of parallelism. For each degree, we report the performance in MOPS/s (higher is better) of Baseline and the SCHED_COOP speedup. Mild uses 8x8 threads (1.14 threads per core), Medium uses 14x14 (3.5 threads per core), and High uses 28x28 (14 threads per core). gnu stands for GNU’s OpenMP, llvm for LLVM’s OpenMP, pth for the pthread BLAS backend, and opb for OpenBLAS.

Out	Inn	BLAS	Mild	Medium	High
gnu	llvm	opb	1468, 1.1x	580, 2.36x	347, 3.14x
tbb	llvm	opb	1619, 1.1x	602, 2.47x	363, 3.22x
tbb	gnu	blis	2004, 1.06x	694, 3.23x	358, 4.28x
tbb	pth	blis	1590, 1.34x	356, 6.28x	103, 14.4x
gnu	pth	blis	1413, 1.33x	254, 6.86x	86, 14.7x

the baseline in all cases, particularly in the pth cases. That’s because the BLIS pthread backend does not reuse threads but creates and destroys them for every dgemm call. Instead, USF caches threads transparently and prevents OS noise (see Section 4.3.1). We found that the pthread cache roughly multiplies by 4 the base SCHED_COOP performance. On the other cases, the pthread cache had no effect because run-times reuse pthreads efficiently. Note that the outer "gnu" and inner "llvm" use static builds to prevent symbol collision.

The key takeaway is that even for simple applications, we might have multiple combinations of runtimes, implementations, and compilation options that can quickly complicate our strategies to handle oversubscription. Coordinating all these scenarios manually would entail a huge effort in understanding and adapting each possibility. Instead, SCHED_COOP is capable of managing all these cases transparently, while keeping the user in control.

5.5 Microservices Case Study

This section examines a Python multi-process AI inference benchmark. Python dominates AI development, but its Global Interpreter Lock (GIL) serializes threads, prompting developers to use multiple processes with native parallel libraries instead. We leverage this common pattern to demonstrate SCHED_COOP in an oversubscribed multi-process environment, comparing it against various resource partitioning schemes under the standard Linux scheduler. We focus on CPU-based inference, which is gaining research attention [31, 42, 67, 71, 72] due to CPUs’ ubiquity, flexibility, and cost-effectiveness.

Our benchmark simulates periodic client requests following a Poisson distribution, processed by four Python server processes on a dedicated node. A Gateway process receives requests, simulates planning logic, and forwards them to three inference servers in parallel, blocking until all responses return. Each server runs a distinct model: Meta’s

LLaMA 3.2 (1B)[58], OpenAI’s GPT-2 (124M)[48], and a fine-tuned Meta RoBERTa-large (355M) [24] for sentiment analysis. All models run on PyTorch, which uses efficient C++ libraries and OpenBLAS with OpenMP to bypass GIL limitations. Thus, each user request creates one thread per process (four total), with three threads spawning pthread teams for parallel inference computation. At high request frequencies, overlapping requests cause system oversubscription.

We evaluate the behavior of our benchmark as we increase the request frequency. Each execution generates 28 requests of 128 tokens (~168 words) and simulates a grouping of 8 batches per request. Each model outputs at most 32 tokens (~42 words). We do not limit the maximum number of outer threads servicing requests, but we fix the number of inner BLAS threads used to compute each inference. We calculated the ideal number of inner threads per model by running an isolated strong scalability test with a single request. In particular, LLaMA obtained 5.4s with 28 cores, GPT-2 1.8s with 8 cores, and RoBERTa 1.2s with 8 cores.

We compare the results of running the benchmark under SCHED_COOP with four baseline scenarios. b1-none employs no resource partitioning. b1-eq splits equally its cores among all Python inference servers but for the Gateway, which uses 2 cores. b1-opt uses an optimized partitioning scheme based on the scalability results obtained earlier to proportionally assign cores to the most demanding servers minus two cores for the Gateway server. The optimized partitioning splits the node’s 112 cores in 71 cores for LLaMA (64%), 23 cores for GPT-2 (21%), and 16 cores for RoBERTa (14%). Note that even with this resource partition, there is still oversubscription within each partition as the request frequency increases. However, partitioning helps limit overall oversubscription. b1-none-seq uses no partitioning and avoid oversubscription by computing inferences sequentially (without OpenMP parallelism).

All tests use a nice priority of 0 for the Gateway server (high priority) versus a priority of 20 (less priority) for the inference servers in order to attend and respond user requests as quickly as possible. SCHED_COOP does not need nice priorities, but we assign them to consider the noise caused by other system processes.

Figure 4 (top) shows the throughput and latency obtained for SCHED_COOP and the three Baselines. As expected, the static partition in b1-eq obtains the worst results for both throughput and latency because of the load imbalance among partitions. b1-opt fixes the imbalance to some extent; although, as the rate increases, the LLaMA server becomes a bottleneck and stalls the overall backend. b1-none gives full control to the Linux scheduler and obtains remarkable results although it eventually collapses because of synchronization noise worsened by oversubscription. b1-none-seq achieves stable latencies across all request rates and high throughput at the highest rates due to the absence of oversubscription

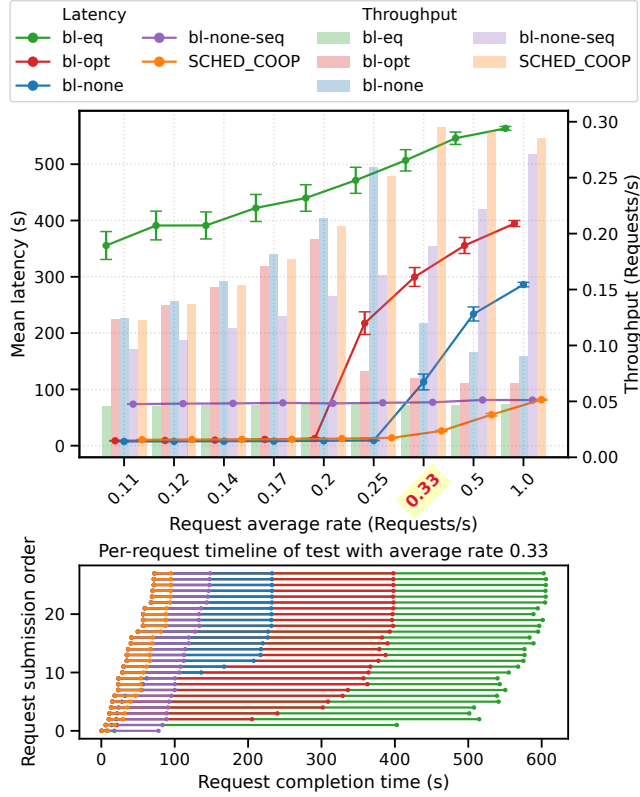


Figure 4. Evaluation of the agentic microservices benchmark. The top figure shows latency and throughput as the frequency of requests increases. The bottom figure shows the detailed per-request start and end time of the test case with rate 0.33.

noise. However, since servers lack OpenMP parallelism, individual inference times are higher—especially at lower request rates where most cores remain idle. Finally, SCHED_COOP achieves both low latency and sustained throughput for the whole range of rates. Figure 4 (bottom) shows per-request latencies for the run with 0.33 average rate, where bl-none collapses. As ongoing requests accumulate and oversubscription increases, the Linux fair scheduler keeps assigning CPU time evenly to all threads. This causes the requests to progress evenly, including most OpenMP busy-wait barriers and their associated noise, which remain constant until all requests finish nearly simultaneously. Instead, SCHED_COOP can process requests faster, finish them earlier, and prevent the noise generated by an excessive accumulation of yielding threads, achieving up to 2.4x with respect to bl-none.

As a side note, it would be reasonable to think that non-preemptive schedulers could favor throughput over latency. However, in this scenario, the constant synchronizations of the BLAS libraries act as scheduling points that allow all participating processes to progress despite the lack of involuntary preemptions.

5.6 LAMMPS and DeePMD-kit Case Study

This section studies the LAMMPS MD simulator coupled with DeePMD-kit, a deep-learning software that models interatomic potential energies and force fields. Classic quantum mechanical calculations, such as density functional theory (DFT), provide highly accurate results but are computationally prohibitive for large-scale simulations. Specialized AI models trained on data from these first-principles methods can achieve comparable accuracy at a fraction of the computational cost [37, 62]. LAMMPS can use MPI to decompose the physical simulation domain into spatial regions, where each MPI rank computes forces and energies for atoms within its assigned region and exchanges boundary data with neighboring ranks. Within each rank, DeePMD-kit runs inferences with PyTorch, which internally leverages an OpenMP-parallelized OpenBLAS for its computational kernels.

The particular atomic spatial distribution and MPI partitioning of a simulation might become imbalanced, as moving particles cross inter-rank boundaries and redistribute each rank workload dynamically at runtime. Hence, it is not trivial to efficiently distribute processes and threads into cores to maximizing resource usage. However, it is a common technique to execute multiple independent simulations or ensembles to explore different paths [14, 41], which are usually run one after the other or in different jobs. However, ensembles might add another level of parallelism that we can leverage to improve performance by filling the gaps of one simulation with another. However, at the same time, this further exacerbates the risk of scheduling interference between ensembles, processes, and threads, while under heavy computation phases.

We analyze multiple execution and oversubscription scenarios of a two LAMMPS ensemble setup with hybrid MPI and OpenMP parallelization. In Exclusive, ensembles run one after the other, use 56 MPI processes with 2 OpenMP threads each, and are pinned to disjoint core sets. The Co-execution scenarios are similar, but the ensembles run in parallel. In Co-location, we avoid oversubscription by reducing the number of MPI processes to 28 (we keep 2 OpenMP threads per rank), and we pin each rank to a disjoint core set. In SCHED_COOP the ensembles are managed by our scheduler and use 56 MPI and 2 OpenMP each. All configurations but Exclusive feature two variations: In the “socket” variant the processes of each ensemble are distributed equally among the two sockets of the machine (half in one socket, and half in the other). Instead, in the “node” variant, each ensemble is confined in a different socket.

We trained a standard DeePMD-kit *se_e2_a* model with a descriptor size of [25, 50, 100], a fitting network of [240, 240, 240], and 6 Å cutoff. The simulation creates 20 K CH_4 molecules (5 atoms each, 100 K atoms in total) created in a box of $292.4 \times 292.4 \times 292.4$ Å and distributed in a set of 14

interleaved dense and sparse regions along the x-axis, where dense regions contains 90% of the atoms, while sparse regions the remaining 10%. The 3D simulation box is split between ranks along the x-axis, creating an unbalanced environment. We keep a low temperature of 5 degrees Kelvin to keep atom velocities low. We run each simulation for 100 steps.

Figure 5a shows the evaluation results for all configurations, and Figure 5b compares the total memory bandwidth (reads and writes) observed in the evaluation node during an execution of each configuration. In the Exclusive mode, individual ensembles attain up to 106 Katom-step/s while in the other configurations, at most 60 Katoms-setp/s is achieved. That is because there is no interference with other runtimes and each rank can use the full memory bandwidth. However, as shown in Figure 5b, a single ensemble is incapable of leveraging the whole bandwidth at any point. Moreover, the initialization of each ensemble is sequential (shown as the initial and middle bandwidth valleys), and its cost must be paid per each ensemble; in the other scenarios, the ensemble initialization can be done in parallel, more parallelism is available to exploit the machine resources, and the aggregated Katom-step/s of all ensembles surpass Exclusive.

Still, both co-location scenarios suffer from heavy load imbalance which drives half the caches and cores underused. Besides, the amount of data needed to be kept in the caches with respect to Exclusive doubles because both ensembles are running in parallel. The socket variant gets the worst performance because it also needs to pay the cost of inter-processor communications.

The co-execution scenarios can exploit all cores and caches because 1) each ensemble exposes in total as many threads as cores, and 2) threads can migrate between cores. However, at the same time, the effect of oversubscription and busy wait barriers of both MPICH and OpenBLAS limits their benefits.

Finally, the SCHED_COOP variants present none of the previous limitations but all their advantages: Running both ensembles in parallel enables more parallel workload to compensate the imbalance, threads can migrate to utilize all resources, and busy waiting is no longer stalling cores. In consequence, SCHED_COOP attains both the highest Katoms-setp/s and memory bandwidth usage, for up to 4% speedup compared with the coexecution socket scenario. Still, because USF does not provide support for I/O syscalls, most blocking MPI communications stall cores until they complete. In the future, we will study the integration of these operations to further improve these cases.

5.7 Takeaways

From our evaluation, we have learned the following lessons: (1) Applications that combine multiple runtimes are generally complex to tune and debug. Under oversubscription, these applications obtain huge gains from minor code adaptations to avoid busy-wait barriers. (2) Application coexecution is advantageous to maximize resource usage when

the involved applications exhibit different degrees of parallelism, but its potential is quickly undermined by the negative effects of preemptive schedulers under oversubscription. (3) SCHED_COOP can manage both scenarios and, unlike similar approaches, it works transparently, requires no elevated privileges, and is fully compatible with TLS. (4) USF policies are compatible with any application that relies on standard synchronization APIs provided by glibc, but SCHED_COOP is particularly useful for oversubscribed scenarios with reasonable granularity settings. (5) Cooperative user-space schedulers can harness the potential in oversubscription scenarios of modern applications, previously avoided due to performance degradation. (6) Its main caveat is custom busy-wait barriers. However, we have shown that they are simple to identify and fix.

6 Related Work

Several works have studied the challenges of coordinating both runtime and kernel schedulers [32, 33]. Such challenge grows even larger when multiple different runtimes participate within a single process and oversubscribe the system [65], such as Python and its high modularity [35]. These approaches typically cap thread counts, enforce a single runtime, or extend runtimes to coordinate with each other.

It is possible to exploit application imbalance by coexecuting multiple processes on the same node to fill the gaps of one process with another. This has been applied in cloud elasticity [25], addressing issues like OS scalability and busy-waiting, though without considering preemptions in critical regions—particularly problematic for parallel runtimes. In HPC, oversubscribing MPI processes can help overlap communication with computation [3], but managing multiple runtimes makes this harder. Other work [28] showed that limited oversubscription benefits coarse-grained workloads, yet modern many-core systems are more vulnerable to interrupted synchronizations under system-level preemptive schedulers [12, 56]. A common alternative is resource partitioning: Arachne [47] and DLB [18] dynamically assign cores to processes at runtime, achieving good results but requiring substantial application or runtime changes to use their APIs.

Recently, there is renewed interest in coexecuting applications as HPC workloads use external Artificial Intelligence (AI) services running as separate processes [38] (AI-out-HPC). However, finding a unified solution to coordinate all involved components and disciplines is challenging. Instead, tackling the problem at the system-level scheduler has the advantage of providing a transparent solution with the potential of greatly simplifying the integration of both worlds.

In that regard, the Linux community has advanced user-space scheduling with the Extensible Scheduler Class (sched_ext) [11], which allows schedulers to be implemented as eBPF [19] programs. However, decisions are limited to predefined kernel hooks, eBPF code cannot use

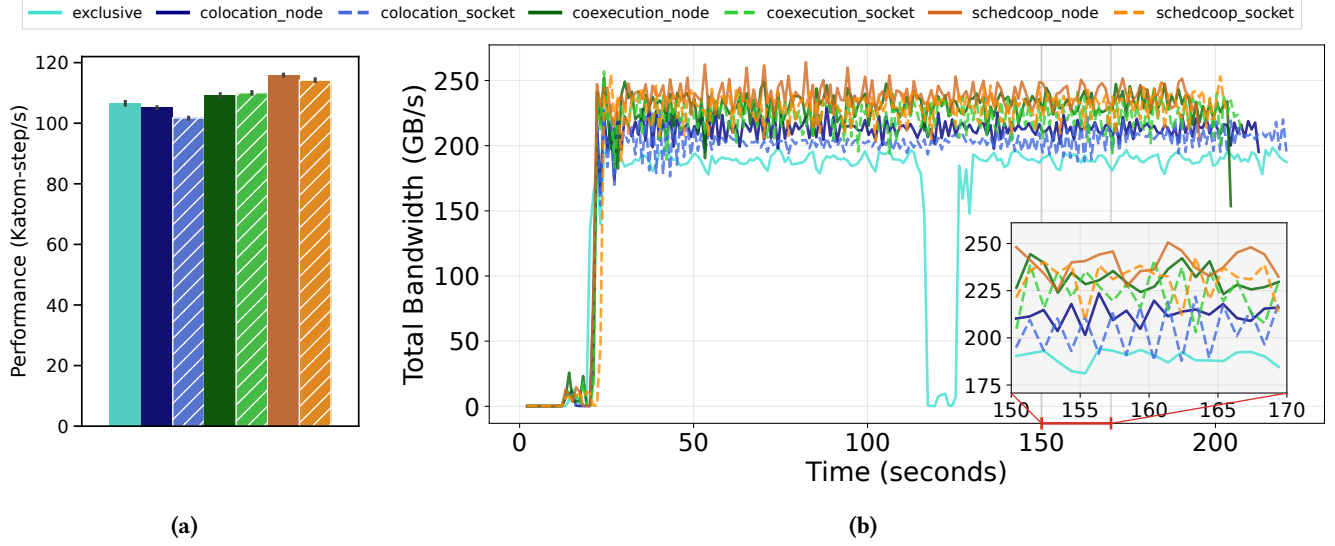


Figure 5. LAMMPS and DeePMD-Kit evaluation with two ensembles for several configurations. All ensembles use 56 MPI and 2 OpenMP each, except co-location versions that limit MPI ranks to 28 in order to avoid oversubscription. Figure a) shows performance. Figure b) shows measured total main memory bandwidth for both reads and writes. The inner plot in Figure b) shows a zoomed area of 20s starting at second 150. Average bandwidth in GB/s per scenario is: Exclusive 165.36, colocation_node 193.09, colocation_socket 185.80, coexecution_node 208.17, coexecution_socket 201.99, schedcoop_node 214.78, schedcoop_socket 205.64.

floating point and requires compile-time loop bounds, and user privileges are needed to load programs. Consequently, Google’s ghOSt [27] and the F4 framework [73] propose user-space integrated schedulers without eBPF restrictions, but their reliance on extensive kernel modifications hinders adoption in Linux. Syrup [30] has similar limitations because it relies on ghOSt or eBPF. Purely cooperative user-space schedulers are commonly used to manage User-Level Threads (ULTs) [8, 29, 52], but their lack of TLS support restricts their applicability to generic workloads without prior adaptation.

To the best of our knowledge, USF is the first seamless user-space scheduler framework that supports multiple processes and that works within existing Linux environments through glibc extensions. While its SCHED_COOP policy addresses the fundamental problems of LHP and LWP in oversubscribed environments while keeping TLS support.

7 Conclusions and Future Work

In this paper, we present USF, a framework to develop seamless user-space scheduling policies, and SCHED_COOP, its default cooperative policy. Both are entirely implemented in user-space by extending glibc, require no Linux kernel extensions, and support TLS. Our framework allows the development of ad-hoc scheduling policies and provides the means to define how standard blocking APIs behave. It supports any application and runtime that relies on standard glibc’s synchronization primitives, although custom busy

wait barriers need to be adapted. We have extensively tested SCHED_COOP with both multi-process and multi-runtime scenarios and showed that it is effective at managing oversubscription, achieving up to 2.4x.

Future work includes adding support for blocking Linux I/O via io_uring (as in TASIO [50]); extend the set of supported blocking calls to include futex; expose additional glibc APIs to interface with the USF core; explore an in-kernel implementation of SCHED_COOP and compare it with the glibc version; evaluate signal-based preemption in USF; and evaluate new custom scheduling policies targeting different workloads such as mixed criticality systems, or interactive workloads.

Acknowledgments

This work was partially supported by the Generalitat de Catalunya (contract 2021-SGR-01007) and the Spanish Government through the Severo Ochoa Program (CEX2021-001148-S/MCIN/AEI/10.13039/501100011033), and it is part of the ST4HPC (PID2023-147979NB-C21) research project, funded by MICIU/AEI/10.13039/501100011033 and by FEDER, UE. This paper is also co-financed by the Barcelona Zettaescale Laboratory, promoted by the Ministry for Digital Transformation and the Civil Service, within the framework of the Recovery, Transformation and Resilience Plan - Funded by the European Union - NextGenerationEU. We thank the administrators of BSC’s Jungle cluster for their technical support.

References

- [1] Cédric Augonnet, Samuel Thibault, Raymond Namyst, and Pierre-André Wacrenier. 2009. StarPU: A Unified Platform for Task Scheduling on Heterogeneous Multicore Architectures. In *Euro-Par 2009 Parallel Processing*, Henk Sips, Dick Epema, and Hai-Xiang Lin (Eds.). Springer, Berlin, Heidelberg, 863–874. doi:10.1007/978-3-642-03869-3_80
- [2] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. 2012. Legion: Expressing locality and independence with logical regions. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–11. doi:10.1109/SC.2012.71 ISSN: 2167-4337.
- [3] Jan Bierbaum, Maksym Planeta, and Hermann Härtig. 2022. Towards Efficient Oversubscription: On the Cost and Benefit of Event-Based Communication in MPI. In *2022 IEEE/ACM International Workshop on Runtime and Operating Systems for Supercomputers (ROSS)*. 1–10. doi:10.1109/ROSS56639.2022.00007
- [4] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. 1995. Cilk: an efficient multithreaded runtime system. *SIGPLAN Not.* 30, 8 (Aug. 1995), 207–216. doi:10.1145/209937.209958
- [5] Alex D. Breslow, Leo Porter, Ananta Tiwari, Michael Laurenzano, Laura Carrington, Dean M. Tullsen, and Allan E. Snaveley. 2016. The case for colocation of high performance computing workloads. *Concurr. Comput. : Pract. Exper.* 28, 2 (Feb. 2016), 232–251. doi:10.1002/cpe.3187
- [6] Programming Models Group BSC. 2024. Nanos6 Runtime Main Repository. <https://github.com/bsc-pm/nanos6> Accessed: 2024-04-22.
- [7] Qinglei Cao, Yu Pei, Kadir Akbudak, George Bosilca, Hatem Ltaief, David Keyes, and Jack Dongarra. 2021. Leveraging ParSEC Runtime Support to Tackle Challenging 3D Data-Sparse Matrix Problems. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 79–89. doi:10.1109/IPDPS49936.2021.00017 ISSN: 1530-2075.
- [8] Arnau Cinca, Aleix Roca, Kevin Sala, Raúl Peñacoba, David Álvarez, and Vicenç Beltran. 2025. Enhancing OmpSs-2 Suspendable Tasks by Combining Operating System and User-Level Threads with C++ Coroutines. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 67–80. doi:10.1109/IPDPS64566.2025.00015 ISSN: 1530-2075.
- [9] Linux community. [n. d.]. EEVDF Scheduler — The Linux Kernel documentation. <https://docs.kernel.org/scheduler/sched-eevdf.html>
- [10] Linux Community. 2025. CFS Scheduler — The Linux Kernel documentation. <https://docs.kernel.org/scheduler/sched-design-CFS.html>
- [11] Linux Community. 2025. Extensible Scheduler Class — The Linux Kernel documentation. <https://docs.kernel.org/scheduler/sched-ext.html>
- [12] Dave Dice and Alex Kogan. 2019. Avoiding Scalability Collapse by Restricting Concurrency. arXiv:1905.10818 [cs.OS]
- [13] Dave Dice and Alex Kogan. 2021. Intra-process Caching and Reuse of Threads. arXiv preprint arXiv:2105.07497 (2021).
- [14] Tu Mai Anh Do, Loïc Pottier, Rafael Ferreira da Silva, Silvina Caínolores, Michela Taufer, and Ewa Deelman. 2023. Performance assessment of ensembles of in situ workflows under resource constraints. *Concurrency and Computation: Practice and Experience* 35, 20 (2023), e7111.
- [15] Eelco Dolstra. 2006. *The Purely Functional Software Deployment Model*. Ph.D. Dissertation. Utrecht University, Utrecht, The Netherlands. Advisor(s) Eelco Visser and S. Doaitse Swierstra.
- [16] Alejandro Duran, Marc González, and Julita Corbalán. 2005. Automatic thread distribution for nested parallelism in OpenMP. In *Proceedings of the 19th annual international conference on Supercomputing (ICS '05)*. Association for Computing Machinery, New York, NY, USA, 121–130. doi:10.1145/1088149.1088166
- [17] Jorge Ejarque, Rosa M. Badia, Loïc Albertin, Giovanni Aloisio, Enrico Baglione, Yolanda Becerra, Stefan Boschert, Julian R. Berlin, Alessandro D'Anca, Donatello Elia, François Exertier, Sandro Fiore, José Flich, Arnau Folch, Steven J. Gibbons, Nikolay Koldunov, Francesc Lordan, Stefano Lorito, Finn Løvholt, Jorge Macías, and Manuela Volpe. 2022. Enabling dynamic and intelligent workflows for HPC, data analytics, and AI convergence. (Sept. 2022). doi:10.13039/501100002347 Accepted: 2023-04-18T11:30:03Z Publisher: Elsevier.
- [18] Marta Garcia Gasulla. 2017. *Dynamic load balancing for hybrid applications*. Doctoral thesis. Universitat Politècnica de Catalunya. doi:10.5821/dissertation-2117-108227 Accepted: 2017-10-02T00:31:03Z Publication Title: TDX (Tesis Doctorals en Xarxa).
- [19] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Høiland-Jørgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunström. 2024. The eBPF Runtime in the Linux Kernel. doi:10.48550/arXiv.2410.00026 arXiv:2410.00026 [cs].
- [20] Pawel Gepner. 2021. Machine Learning and High-Performance Computing Hybrid Systems, a New Way of Performance Acceleration in Engineering and Scientific Applications. In *2021 16th Conference on Computer Science and Intelligence Systems (FedCSIS)*. 27–36. doi:10.15439/2021F004
- [21] GNU. [n. d.]. The GNU C Library - GNU Project - Free Software Foundation. <https://www.gnu.org/software/libc/>
- [22] Kazushige Goto and Robert A van de Geijn. 2008. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software (TOMS)* 34, 3 (2008), 1–25.
- [23] William Gropp, Ewing (Rusty) Lusk, Rajeev Thakur, Pavan Balaji, Thomas Gillis, Yanfei Guo, Rob Latham, Ken Raffenetti, and Hui Zhou. 2023. MPICH. [Computer Software] <https://doi.org/10.1157/dc.20200514.13> doi:10.1157/dc.20200514.13
- [24] Jochen Hartmann, Mark Heitmann, Christian Siebert, and Christina Schamp. 2023. More than a Feeling: Accuracy and Application of Sentiment Analysis. *International Journal of Research in Marketing* 40, 1 (2023), 75–87. doi:10.1016/j.ijresmar.2022.05.005
- [25] Hang Huang, Jia Rao, Song Wu, Hai Jin, Hong Jiang, Hao Che, and Xiaofeng Wu. 2020. Towards Exploiting CPU Elasticity via Efficient Thread Oversubscription. In *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing (Virtual Event, Sweden) (HPDC '21)*. Association for Computing Machinery, New York, NY, USA, 215–226. doi:10.1145/3431379.3460641
- [26] E. A. Huerta, Asad Khan, Edward Davis, Colleen Bushell, William D. Gropp, Daniel S. Katz, Volodymyr Kindratenko, Seid Koric, William T. C. Kramer, Brendan McGinty, Kenton McHenry, and Aaron Saxton. 2020. Convergence of artificial intelligence and high performance computing on NSF-supported cyberinfrastructure. *Journal of Big Data* 7, 1 (Oct. 2020), 88. doi:10.1186/s40537-020-00361-2
- [27] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, and Christos Kozyrakis. 2021. ghOST: Fast & Flexible User-Space Delegation of Linux Scheduling. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 588–604. doi:10.1145/3477132.3483542
- [28] Costin Iancu, Steven Hofmeyr, Filip Blagojević, and Yili Zheng. 2010. Oversubscription on multicore processors. In *2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*. 1–11. doi:10.1109/IPDPS.2010.5470434 ISSN: 1530-2075.
- [29] Shintaro Iwasaki, Abdelhalim Amer, Kenjiro Taura, Sangmin Seo, and Pavan Balaji. 2019. BOLT: Optimizing OpenMP Parallel Regions with User-Level Threads. In *2019 28th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 29–42. doi:10.1109/PACT.2019.00011 ISSN: 2641-7936.
- [30] Kostis Kaffes, Jack Tigar Humphries, David Mazières, and Christos Kozyrakis. 2021. Syrup: User-Defined Scheduling Across the Stack. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 605–620. doi:10.1145/3477132.3483548
- [31] Hyungyo Kim, Nachuan Wang, Qirong Xia, Jinghan Huang, Amir Yazdanbakhsh, and Nam Sung Kim. 2025. LIA: A Single-GPU LLM

- Inference Acceleration with Cooperative AMX-Enabled CPU-GPU Computation and CXL Offloading. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 544–558. doi:10.1145/3695053.3731092
- [32] Jonas H. Müller Korndörfer, Ahmed Eleliemy, Ali Mohammed, and Florina M. Ciorba. 2022. LB4OMP: A Dynamic Load Balancing Library for Multithreaded Applications. *IEEE Transactions on Parallel and Distributed Systems* 33, 4 (April 2022), 830–841. doi:10.1109/TPDS.2021.3107775
- [33] Jonas H. Müller Korndörfer, Ahmed Eleliemy, Osman Seckin Simsek, Thomas Ilsche, Robert Schöne, and Florina M. Ciorba. 2023. How Do OS and Application Schedulers Interact? An Investigation with Multithreaded Applications. In *Euro-Par 2023: Parallel Processing*, José Cano, Marios D. Dikaiakos, George A. Papadopoulos, Miquel Pericàs, and Rizos Sakellariou (Eds.). Springer Nature Switzerland, Cham, 214–228. doi:10.1007/978-3-031-39698-4_15
- [34] Kin Long Kelvin Lee and Nalini Kumar. 2023. Artificial Intelligence for Scientific Discovery at High-Performance Computing Scales. *Computer* 56, 4 (April 2023), 116–122. doi:10.1109/MC.2023.3241692
- [35] Anton Malakhov, David Liu, Anton Gorshkov, and Terry Wilmarth. 2018. Composable Multi-Threading and Multi-Processing for Numeric Libraries. *scipy* (June 2018). doi:10.25080/Majora-4af1f417-003
- [36] man page for shm_overview. [n. d.]. shm overview. https://man7.org/linux/man-pages/man7/shm_overview.7.html. Accessed: 2021-08-07.
- [37] Raidel Martin-Barrios, Edisel Navas-Conyedo, Xuyi Zhang, Yunwei Chen, and Jorge Gulín-González. 2024. An overview about neural networks potentials in molecular dynamics simulation. *International Journal of Quantum Chemistry* 124, 11 (2024), e27389.
- [38] Andre Merzky, Mikhail Titov, Matteo Turilli, Ozgur Kilic, Tianle Wang, and Shantenu Jha. 2025. Scalable Runtime Architecture for Data-driven, Hybrid HPC and ML Workflow Applications. doi:10.48550/arXiv.2503.13343 arXiv:2503.13343 [cs].
- [39] Antoni Navarro. 2025. bsc-pm/nodes. <https://github.com/bsc-pm/nodes> original-date: 2023-04-18T09:43:48Z.
- [40] Poornima Nookala, Kyle Chard, and Ioan Raicu. 2023. X-Openmp – Extreme Fine-Grained Tasking Using Lock-Less Work Stealing. doi:10.2139/ssrn.4639600
- [41] John Ossyra, Ada Sedova, Arnold Tharrington, Frank Noé, Cecilia Clementi, and Jeremy C Smith. 2019. Porting adaptive ensemble molecular dynamics workflows to the summit supercomputer. In *International Conference on High Performance Computing*. Springer, 397–417.
- [42] Daon Park and Bernhard Egger. 2024. Improving Throughput-oriented LLM Inference with CPU Computations. In *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques (PACT '24)*. Association for Computing Machinery, New York, NY, USA, 233–245. doi:10.1145/3656019.3676949
- [43] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [44] Josep M. Perez, Vicenç Beltran, Jesus Labarta, and Eduard Ayguadé. 2017. Improving the Integration of Task Nesting and Dependencies in OpenMP. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 809–818. doi:10.1109/IPDPS.2017.69
- [45] James C. Phillips, Rosemary Braun, Wei Wang, James Gumbart, Emad Tajkhorshid, Elizabeth Villa, Christophe Chipot, Robert D. Skeel, Laxmikant Kale, and Klaus Schulten. 2005. Scalable molecular dynamics with NAMD. *Journal of Computational Chemistry* 26, 16 (2005), 1781–1802. doi:10.1002/jcc.20289 _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/jcc.20289>.
- [46] Steven J Plimpton, Axel Kohlmeyer, Aidan P Thompson, Stan G Moore, and Richard Berger. 2007. LAMMPS: large-scale atomic/molecular massively parallel simulator. *Zenodo* (2007).
- [47] Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John Ousterhout. 2018. Arachne: core-aware thread management. In *Proceedings of the 13th USENIX conference on Operating Systems Design and Implementation (OSDI'18)*. USENIX Association, USA, 145–160.
- [48] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. (2019).
- [49] James Reinders. 2007. *Intel threading building blocks: outfitting C++ for multi-core processor parallelism*. " O'Reilly Media, Inc."
- [50] Aleix Roca Nonell, Vicenç Beltran Querol, and Sergi Mateo Bellido. 2019. Introducing the Task-Aware Storage I/O (TASIO) Library. In *OpenMP: Conquering the Full Hardware Spectrum*, Xing Fan, Bronis R. de Supinski, Oliver Sinnen, and Nasser Giacaman (Eds.). Springer International Publishing, Cham, 274–288. doi:10.1007/978-3-030-28596-8_19
- [51] Kevin Sala, Alejandro Rico, and Vicenç Beltran. 2020. Towards Data-Flow Parallelization for Adaptive Mesh Refinement Applications. In *2020 IEEE International Conference on Cluster Computing (CLUSTER)*. 314–325. doi:10.1109/CLUSTER49012.2020.00042 ISSN: 2168-9253.
- [52] Sangmin Seo, Abdelhalim Amer, Pavan Balaji, Cyril Bordage, George Bosilca, Alex Brooks, Philip Carns, Adrián Castelló, Damien Genet, Thomas Herault, Shintaro Iwasaki, Prateek Jindal, Laxmikant V. Kalé, Sriram Krishnamoorthy, Jonathan Lifflander, Huiwei Lu, Esteban Meneses, Marc Snir, Yanhua Sun, Kenjiro Taura, and Pete Beckman. 2018. Argobots: A Lightweight Low-Level Threading and Tasking Framework. *IEEE Transactions on Parallel and Distributed Systems* 29, 3 (March 2018), 512–526. doi:10.1109/TPDS.2017.2766062
- [53] Tyler M. Smith, Robert A. van de Geijn, Mikhail Smelyanskiy, Jeff R. Hammond, and Field G. Van Zee. 2014. Anatomy of High-Performance Many-Threaded Matrix Multiplication. In *28th IEEE International Parallel & Distributed Processing Symposium (IPDPS 2014)*. <https://doi.org/10.1109/IPDPS.2014.110>
- [54] Ion Stoica and Hussein Abdel-Wahab. 1995. Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation. *Old Dominion Univ., Norfolk, VA, Tech. Rep. TR-95-22* (1995).
- [55] Sureshini, Priyanka Sharma, Mohit Ved, Janaki Chintalapati, and Supriya N. Pal. 2021. Big Data Analytics and Machine Learning Technologies for HPC Applications. In *Evolving Technologies for Computing, Communication and Smart World*, Pradeep Kumar Singh, Arti Noor, Maheshkumar H. Kolekar, Sudeep Tanwar, Raj K. Bhatnagar, and Shaweta Khanna (Eds.). Springer, Singapore, 411–424. doi:10.1007/978-981-15-7804-5_31
- [56] Boris Teabe, Vlad Nitu, Alain Tchana, and Daniel Hagimont. 2017. The Lock Holder and the Lock Waiter Pre-Emption Problems: Nip Them in the Bud Using Informed Spinlocks (I-Spinlock). In *Proceedings of the Twelfth European Conference on Computer Systems* (Belgrade, Serbia) (*EuroSys '17*). Association for Computing Machinery, New York, NY, USA, 286–297. doi:10.1145/3064176.3064180
- [57] Aidan P Thompson, H Metin Aktulga, Richard Berger, Dan S Bolinteanu, W Michael Brown, Paul S Crozier, Pieter J In't Veld, Axel Kohlmeyer, Stan G Moore, Trung Dac Nguyen, et al. 2022. LAMMPS-a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer physics communications* 271 (2022), 108171.
- [58] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [59] Field G. Van Zee and Robert A. van de Geijn. 2015. BLIS: A Framework for Rapidly Instantiating BLAS Functionality. *ACM Trans. Math. Software* 41, 3 (June 2015), 14:1–14:33. <https://doi.acm.org/10.1145/2764454>
- [60] Ioannis Vardas, Sascha Hunold, Philippe Swartvagher, and Jesper Larsen Träff. 2024. Exploring Mapping Strategies for Co-allocated

- HPC Applications. In *Euro-Par 2023: Parallel Processing Workshops*, Demetris Zinalipour, Dora Blanco Heras, George Pallis, Herodotos Herodotou, Demetris Trihinas, Daniel Balouek, Patrick Diehl, Terry Cojean, Karl Förlinger, Maja Hanne Kirkeby, Matteo Nardelli, and Pierangelo Di Sanzo (Eds.). Springer Nature Switzerland, Cham, 271–276. doi:10.1007/978-3-031-48803-0_31
- [61] Raúl Peñacoba Veigas, David Álvarez Robert, Vincent A Arcila Larrea, Antoni Navarro Muñoz, and Vicenç Beltran Querol. 2025. Supporting OpenMP free-agents by leveraging nOS-V. In *Workshop on Asynchronous Many-Task Systems and Applications*. Springer, 95–107.
- [62] Han Wang, Linfeng Zhang, Jiequn Han, and Weinan E. 2018. DeepPMD-kit: A deep learning package for many-body potential energy representation and molecular dynamics. *Comput. Phys. Comm.* 228 (jul 2018), 178–184. doi:10.1016/j.cpc.2018.03.016
- [63] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45. <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [64] Qingqing Xiong, Emre Ates, Martin C. Herbordt, and Ayse K. Coskun. 2018. Tangram: Colocating HPC Applications with Oversubscription. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*. 1–7. doi:10.1109/HPEC.2018.8547644 ISSN: 2377-6943.
- [65] Yonghong Yan, Jeff R. Hammond, Chunhua Liao, and Alexandre E. Eichenberger. 2016. A Proposal to OpenMP for Addressing the CPU Oversubscription Challenge. In *OpenMP: Memory, Devices, and Tasks*, Naoya Maruyama, Bronis R. de Supinski, and Mohamed Wahib (Eds.). Springer International Publishing, Cham, 187–202. doi:10.1007/978-3-319-45550-1_14
- [66] Chenle Yu, Sara Royuela, and Eduardo Quiñones. 2023. Taskgraph: A Low Contention OpenMP Tasking Framework. *IEEE Trans. Parallel Distrib. Syst.* 34, 8 (Aug. 2023), 2325–2336. doi:10.1109/TPDS.2023.3284219
- [67] Renchao Yu. 2024. Application of CPU in AI and Machine Learning. In *Proceedings of the International Conference on Decision Science & Management (ICDSM '24)*. Association for Computing Machinery, New York, NY, USA, 221–224. doi:10.1145/3686081.3686118
- [68] Felipe Vieira Zacarias, Vinicius Petrucci, Rajiv Nishtala, Paul Carpenter, and Daniel Mossé. 2021. Intelligent colocation of HPC workloads. *J. Parallel and Distrib. Comput.* 151 (May 2021), 125–137. doi:10.1016/j.jpdc.2021.02.010
- [69] Jinzhe Zeng, Duo Zhang, Denghui Lu, Pinghui Mo, Zeyu Li, Yixiao Chen, Marián Rynik, Li'ang Huang, Ziyao Li, Shaochen Shi, Yingze Wang, Haotian Ye, Ping Tuo, Jiabin Yang, Ye Ding, Yifan Li, Davide Tisi, Qiyu Zeng, Han Bao, Yu Xia, Jiameng Huang, Koki Muraoka, Yibo Wang, Junhan Chang, Fengbo Yuan, Sigbjørn Løland Bore, Chun Cai, Yinnian Lin, Bo Wang, Jiayan Xu, Jia-Xin Zhu, Chenxing Luo, Yuzhi Zhang, Rhys E A Goodall, Wenshuo Liang, Anurag Kumar Singh, Sikai Yao, Jingchao Zhang, Renata Wentzcovitch, Jiequn Han, Jie Liu, Weile Jia, Darrin M York, Weinan E, Roberto Car, Linfeng Zhang, and Han Wang. 2023. DeepPMD-kit v2: A software package for deep potential models. *J. Chem. Phys.* 159 (2023), 054801. Issue 5. doi:10.1063/5.0155600
- [70] Jinzhe Zeng, Duo Zhang, Anyang Peng, Xiangyu Zhang, Sensen He, Yan Wang, Xinzijian Liu, Hangrui Bi, Yifan Li, Chun Cai, Chengqian Zhang, Yiming Du, Jia-Xin Zhu, Pinghui Mo, Zhengtao Huang, Qiyu Zeng, Shaochen Shi, Xuejian Qin, Zhaoxi Yu, Chenxing Luo, Ye Ding, Yun-Pei Liu, Ruosong Shi, Zhenyu Wang, Sigbjørn Løland Bore, Junhan Chang, Zhe Deng, Zhaohan Ding, Siyuan Han, Wan-run Jiang, Guolin Ke, Zhaoqing Liu, Denghui Lu, Koki Muraoka, Hananeh Oliaei, Anurag Kumar Singh, Haohui Que, Weihong Xu, Zhangmancang Xu, Yong-Bin Zhuang, Jiayu Dai, Timothy J. Giese, Weile Jia, Ben Xu, Darrin M. York, Linfeng Zhang, and Han Wang. 2025. DeepPMD-kit v3: A Multiple-Backend Framework for Machine Learning Potentials. *J. Chem. Theory Comput.* 21, 9 (2025), 4375–4385. doi:10.1021/acs.jctc.5c00340
- [71] Zhanpeng Zeng, Michael Davies, Pranav Pulijala, Karthikeyan Sankaralingam, and Vikas Singh. 2023. LookupFFN: making transformers compute-lite for CPU inference. In *Proceedings of the 40th International Conference on Machine Learning (ICML '23, Vol. 202)*. JMLR.org, Honolulu, Hawaii, USA, 40707–40718.
- [72] Cheng Zhang, Xingyu Zhu, Longhao Chen, Tingjie Yang, Evens Pan, Guosheng Yu, Yang Zhao, Xiguang Wu, Bo Li, Wei Mao, and Genquan Han. 2025. Enhancing LLM Inference Performance on ARM CPUs Through Software and Hardware Co-Optimization Strategies. *Integrated Circuits and Systems* 2, 2 (May 2025), 49–57. doi:10.23919/ICS.2025.3568404
- [73] Xiaobo Zheng, Zihao Duan, Shiyi Li, Haojun Hu, and Wen Xia. 2024. F4: Fast, Flexible and stable-Fortified User-Space Thread Scheduling Framework. In *2024 International Conference on Networking, Architecture and Storage (NAS)*. 1–8. doi:10.1109/NAS63802.2024.10781349 ISSN: 2835-3323.
- [74] David Álvarez, Kevin Sala, and Vicenç Beltran. 2024. nOS-V: Co-Executing HPC Applications Using System-Wide Task Scheduling. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 312–324. doi:10.1109/IPDPS57955.2024.00035

Received 2025-09-01; accepted 2025-11-10